Document number:	P0905R0
Date:	2018-01-31
Project:	Programming Language C++, Library Evolution Working Group and Evolution Working Group
Reply-to:	*Tomasz Kamiński <tomaszkam at gmail dot com>*
	Herb Sutter <hsutter at microsoft.com>
	Richard Smith <richard at metafoo.co.uk>

Symmetry for spaceship

1. Introduction

This paper proposes to make operator spaceship (`<=>`) symmetric, so that when `a <=> b` is well formed then `b <=> a` should also be well formed and have complementary semantics. This is both for usability purposes, and to make it consistent with the handling of the two-way comparison operators.

2. Revision history

2.1. Revision 0

Initial revision.

3. Motivation and Scope

P0515R3: Consistent comparison proposes that when expression `b < a` is encountered, with `a` and `b` being potentially of different types, the following functions are matched:

1. `operator<(b, a)` and `b.operator<(a)`,
2. `operator<=>(b, a) < 0` and `b.operator<=>(a) < 0`,
3. `0 < operator<=>(a, b)` and `0 < a.operator<=>(b)`.

This guarantees that the class author needs to provide only one definition of `operator<=>` for heterogenous types, and overload resolution itself will try to match it in different configurations to build the desired expression.

Currently, the same mechanism does not work when expression encountered is `b <=> a`, even if `a <=> b` would work. This lack of the built-in symmetry for spaceship operator may lead to surprising behaviour in user code, leading either to compilation errors in seemingly correct programs, or, more importantly, to incorrect runtime behavior.

The remainder of this section illustrates the problem with examples.

3.1. `icase_string` class

Consider the declaration of the comparisons function for the `icase_string` class, representing an sequence of characters for which case-sensitivity is ignored (like file paths in certain operating system):

```
std::weak_ordering operator<=>(icase_string const&, icase_string const&);
std::weak_ordering operator<=>(icase_string const&, std::string_view);
```

With the current specification, above declarations allow two objects:

- `is` of `icase_string`
- `sv` of `std::string_view`

to be compared using the two-way comparison operators (`<`, `>`, ...) regardless of the order of argument. To be specific all of the following expressions are well-formed:

```
is == sv, is != sv, is < sv, is > sv, is <= sv, is >= sv
sv == is, sv != is, sv < is, sv > is, sv <= is, sv >= is
```

However, in the case of the three-way comparison operator, only the expression with original order of arguments will be accepted. Meaning that only the first of the below expressions is well-formed:

```
is <=> sv
sv <=> is // ill-formed
```

This design decision was motivated by the fact that `operator<=>` is not expected to be called by the users, as they naturally use the two-way operators and so use `<=>` only indirectly.

3.2. `optional` example

For example let us consider the following implementation of the spaceship operator for `std::optional` (the following example is based on the code from Barry Rezvin's blog post Implementing the spaceship operator for optional):

```
template<typename T, typename U>
auto operator<=>(optional<T> const& lhs, optional<U> const& rhs)
  -> decltype(*lhs <=> *rhs)
{
   if (lhs.has_value() && rhs.has_value())
     return *lhs <=> *rhs;
   else
     return lhs.has_value() <=> rhs.has_value();
}

template<typename T, typename U>
auto operator<=>(optional<T> const& lhs, U const& rhs)
  -> decltype(*lhs <=> rhs)
{
   if (lhs.has_value())
     return *lhs <=> rhs;
   else
     return strong_ordering::less;
}
```

The above operators are designed to recreate the behaviour of the current `optional` comparisons, that allow an object of the type `optional<T>` to be compared with any object of type `U` or `optional<U>`, if an object of type `T` can be compared with an object of type `U`.

In our example, the user should be able to compare any two of the following objects in either order:

```
icase_string is;
std::string_view sv;
std::optional<icase_string> ois;
std::optional<std::string_view> osv;
```

The above holds in the case of a symmetric invocation on two optionals, because for each `ois @ osv` being:

```
ois == osv, ois != osv, ois < osv, ois > osv, ois <= osv, ois >= osv
```

the synthesised candidate `(ois <=> osv) @ 0` is well formed, as it requires `*ois <=> *osv` (`icase_string` and `std::string`) to be well formed. For the reversed order of arguments i.e. `osv @ oie` being:

```
osv == ois, osv != ois, osv < ois, osv > ois, osv <= ois, osv >= ois
```

the reversed candidate `0 @ (ois <=> osv)` is used for the same reason.

For a symmetric invocation on an optional and an unwrapped object, the only candidate available (`operator<=> (optional<T> const&, U const&)`) always invokes the underlying `<=>` with the wrapped object on the left hand side.

Therefore the invocations in forms `osi @ sv` (rewritten to `(osi <=> sv) @ 0`) and `sv @ osi` (rewritten to `0 @ (osi <=> sv)`) i.e:

```
osi == sv, osi != sv, osi < sv, osi > sv, osi <= sv, osi >= sv
sv == osi, sv != osi, sv < osi, sv > osi, sv <= osi, sv >= osi
```

are well formed, as they lead to `*osi <=> sv` (`icase_string` and `std::string_view`).

In contrast, the invocations in forms `osv @ si` and `si @ osv`, i.e:

```
osv == si, osv != si, osv < si, osv > si, osv <= si, osv >= si
si == osv, si != osv, si < osv, si > osv, si <= osv, si >= osv
```

all ill-formed, because they attempt to invoke `*osv <=> is` (`std::string_view` and `icase_string`).

3.3. `pair` example

Let us consider the following potential extension in form of the heterogeneous three-way comparison operator for `std::pair` types:

```
template<typename T1, typename U1, typename T2, typename U2>
auto operator<=>(std::pair<T1, U1> const& p1, std::pair<T2, U2> const& p2)
 ->  common_comparison_category_t<decltype(p1.first <=> p2.first), decltype(p1.second <=> p2.second)>
{
  if (auto res = p1.first <=> p2.first; res != 0)
    return res;
  return p1.second <=> p2.second;
}
```

The intent of the above operator is to allow pairs containing different types to be compared, if corresponding elements (`first` and `second`) can be compared, providing functionality similar to that already present for the `optional` class template.

However, consider the following declarations of pairs:

```
std::pair<icase_string, std::string_view> p1;
std::pair<std::string_view, icase_string> p2;
```

As the objects of type `icase_string` and `std::string` can be compared witch each other, we can expect that, with the above declarations present, the following expressions will be well-formed:

```
p1 == p2, p1 != p2, p1 < p2, p1 > p2, p1 <= p2, p1 >= p2
p2 == p1, p2 != p1, p2 < p1, p2 > p1, p2 <= p1, p2 >= p1
```

According to the current rules for operator rewrite **none** of them is well-formed: the expression in the form of `p1 == p2` may be interpreted either as:

- `(p1 <=> p2) == 0`
- `0 == (p2 <=> p1)`

In case of the first candidate the expression `p1.first <=> p2.first` (`icase_string` and `std::string_view`) is well-formed, however the three-way comparison of the second elements `p1.second <=> p2.second` (`std::string_view` and `icase_string`) is ill-formed (as invocations of spaceship are not symmetric). For the second candidate the `p2.first <=> p1.first` (`std::string_view` and `icase_string`) is ill-formed again.

In contrast to the `optional` example, where adding the reversed declaration of the mixed operator would address the problem:

```
template<typename T, typename U>
auto operator<=>(T const& lhs, optional<U> const& rhs)
  -> decltype(lhs <=> *rhs)
```

this is insufficient to fix the heterogeneous `pair` comparison. Instead, we would need to edit the original class (`icase_string`) to include a reversed version of the comparison operator:

```
std::weak_ordering operator<=>(std::string_view, icase_string const&);
```

3.4. Incorrect result of the `compare_3way`

In contrast to the previous example, where lack of the symmetry for the invocation of the spaceship operator led to ill-formed code, in this case, the code will compile but produce an incorrect result.

Given the following specification of the `compare_3way` function from 8.7.11 Three-way comparison algorithms ([alg.3way]):

> *Effects*: Compares two values and produces a result of the strongest applicable comparison category type:
>
> - (1.1) Returns `a <=> b` if that expression is well-formed.
> - (1.2) Otherwise, if the expressions `a == b` and `a < b` are each well-formed and convertible to `bool`, returns `strong_ordering::equal` when `a == b` is `true`, otherwise returns `strong_ordering::less` when `a < b` is true, and otherwise returns `strong_ordering::greater`.
> - (1.3) Otherwise, if the expression `a == b` is well-formed and convertible to `bool`, returns `strong_equality::equal` when `a == b` is `true`, and otherwise returns `strong_equality::nonequal`.
> - (1.4) Otherwise, the function is defined as deleted.

The invocation in form `compare_3way(is, sv)` returns an object of type `std::weak_ordering` with value equal to `is <=> sv`. However in case of the reversed order of argument `compare_3way(sv, is)`, the expression `sv <=> is` is ill-formed, so we move to the second point (1.2). In this case the expressions `sv == is` and `sv < is` are well-formed, as they are rewritten to `0 == (is <=> sv)` and `0 < (is <=> sv)` respectively. As a result, we return an object of `std::strong_ordering` with the value matching the inverted value of `is <=> sv`.

Furthermore, consider the case of objects `o1` and `o2` of types `O1` and `O2`, for which the expression `o1 <=> o2` will return `std::partial_order::unordered` and the reverse invocation `o2 <=> o1` will be ill-formed (only `operator<=>(O1, O2)` exists). The invocation of the function `compare_3way(o1, o2)` will return `std::partial_order::unordered`, however if the arguments are reversed `compare_3way(o2, o1)` returns `strong_ordering::greater` (due to the fallback to point 1.2 described above).

In addition all named comparison algorithms (`strong_order`, `weak_order`, `partial_order`, `strong_equal`, `weak_equal`) are prone to the same error caused by the lack of symmetry for three-way operator invocation.

4. Design Decisions

To address these issues, we propose allowing the expression `a <=> b` to find candidates with a reversed order of arguments (`operator<=>(b,a)` or `b.operator<=>(a)`) in addition to the usual set of functions, and if that candidate is selected its returned value is inverted. As a consequence all of the above examples will "just work" without any changes to their code.

To achieve the above goal, we are proposing extending the current language rules for the comparison operators to cover the three-way comparison operator. That means that an expression of the form `a @ b`, with `@` being a relational operator (`==`, `!=`, `<`, `>`, `<=`, `>=`) or **three-way comparison operator (<=>)**, will consider following candidates:

- `a @ b`
- `(a <=> b) @ 0`
- `0 @ (b <=> a)`

where after the rewrite the `@` and `<=>` are interpreted according to usual operator lookup rules, i.e. no recursive rewrites are performed.

Furthermore, we propose to keep the current tie-breakers, that in case of equivalent candidates, prefer:

- `a @ b` over the three-way forms: `(a <=> b) @ 0` and `0 @ (a <=> b)`,
- `(a <=> b) @ 0` over synthesised reversed candidate: `0 @ (b <=> a)`.

Note that in the case of the `<=>`, the `(a <=> b) <=> 0` rewrite will never be used, as in the case when the expression `a <=> b` is well-formed, it will be a worse candidate than `a <=> b`. As a consequence the set of candidates for this operator is de-facto reduced to `a <=> b` and `0 <=> (b <=> a)`.

To complement the above language change, we need to extend to interface of each comparison category type `C` (like `std::strong_ordering`) to include the following functions:

```
C operator<=>(C, unspecified); //c <=> 0, i.e. identity
C operator<=>(unspecified, C); //0 <=> c, i.e. inversion
```

This will basically complete the set of comparison operators between these categories and the literal `0`, which currently only include relational operators.

For an object `c` of a comparison category type, the expression `c <=> 0` is an identity, that returns the value of `c` unchanged, while `0 <=> c` represents an inversion, i.e. returns:

- `C::less` for `c` representing greater-than comparison result (`c > 0`),
- `C::greater` for `c` representing lower-than comparison result (`c < 0`),
- `c` (unchanged) in case of the other values.

Note that for the `strong_equality` and `weak_equality` the inversion is an identity operation, as these objects cannot represent less-than or greater-than results.

5. Proposed Wording

The proposed wording changes refer to N4713 (C++ Working Draft, 2017-11-27).

5.1. Core wording

Change in [over.match.oper] Operators in expressions paragraph 6 as follows:

> The set of candidate functions for overload resolution for some operator `@` is the union of the member candidates, the non-member candidates, and the built-in candidates for that operator `@`. If that operator is a relational ([exp.rel]) ~~or~~, equality ([expr.eq]), or three-way comparison ([expr.spaceship]) operator with operands x and y, then for each member, non-member, or built-in candidate for the operator `<=>`:
>
> - that operator is added to the set of candidate functions for overload resolution if `@` is not `<=>` and `(x <=> y) @ 0` is well-formed using that `operator<=>`; and
> - a synthesized candidate is added to the candidate set where the order of the two parameters is reversed if `0 @ (y <=> x)` is well-formed using that `operator<=>`;
>
> where in each case,
>
> - if `@` is not `<=>`, `operator<=>` candidates are not considered for the recursive lookup of operator `@` and
> - synthesized `operator<=>` candidates are not considered for the recursive lookup of operator `<=>`.

Change in [over.match.oper] Operators in expressions paragraph 8 as follows:

> If an `operator<=>` candidate is selected by overload resolution for an operator `@`, ~~but `@` is not `<=>`,~~ `x @ y` is interpreted as `0 @ (y <=> x)` if the selected candidate is a synthesized candidate with reversed order of parameters, or `(x <=> y) @ 0` ~~otherwise~~if `@` is not `<=>`, using the selected `operator<=>` candidate.

5.2. Library wording

Add the following declarations at the end of the definition of the class in [cmp.weakeq] Class `weak_equality` section.

```
friend constexpr weak_equality operator<=>(weak_equality v, unspecified) noexcept;
friend constexpr weak_equality operator<=>(unspecified, weak_equality v) noexcept;
```

Insert the following at the end of [cmp.weakeq] Class `weak_equality` section.

```
constexpr weak_equality operator<=>(weak_equality v, unspecified) noexcept;
constexpr weak_equality operator<=>(unspecified, weak_equality v) noexcept;
```

Returns: `v`

Add the following declarations at the end of the definition of the class in [cmp.strongeq] Class `strong_equality` section.

```
friend constexpr strong_equality operator<=>(strong_equality v, unspecified) noexcept;
friend constexpr strong_equality operator<=>(unspecified, strong_equality v) noexcept;
```

Insert the following at the end of [cmp.strongeq] Class `strong_equality` section.

```
constexpr strong_equality operator<=>(strong_equality v, unspecified) noexcept;
constexpr strong_equality operator<=>(unspecified, strong_equality v) noexcept;
```

Returns: `v`

Add the following declarations at the end of the definition of the class in [cmp.partialord] Class `partial_ordering` section.

```
friend constexpr partial_ordering operator<=>(partial_ordering v, unspecified) noexcept;
friend constexpr partial_ordering operator<=>(unspecified, partial_ordering v) noexcept;
```

Insert the following at the end of [cmp.partialord] Class `partial_ordering` section.

```
constexpr partial_ordering operator<=>(partial_ordering v, unspecified) noexcept;
```

Returns: `v`

```
constexpr partial_ordering operator<=>(unspecified, partial_ordering v) noexcept;
```

Returns: `v < 0 ? partial_ordering::greater : v > 0 ? partial_ordering::less : v`

Add the following declarations at the end of the definition of the class in [cmp.weakord] Class `weak_ordering` section.

```
friend constexpr weak_ordering operator<=>(weak_ordering v, unspecified) noexcept;
friend constexpr weak_ordering operator<=>(unspecified, weak_ordering v) noexcept;
```

Insert the following at the end of [cmp.weakord] Class `weak_ordering` section.

```
constexpr weak_ordering operator<=>(weak_ordering v, unspecified) noexcept;
```

Returns: `v`

```
constexpr weak_ordering operator<=>(unspecified, weak_ordering v) noexcept;
```

Returns: `v < 0 ? weak_ordering::greater : v > 0 ? weak_ordering::less : v`

Add the following declarations at the end of the definition of the class in [cmp.strongord] Class `strong_ordering` section.

```
friend constexpr strong_ordering operator<=>(strong_ordering v, unspecified) noexcept;
friend constexpr strong_ordering operator<=>(unspecified, strong_ordering v) noexcept;
```

Insert the following at the end of [cmp.strongord] Class `strong_ordering` section.

```
constexpr strong_ordering operator<=>(strong_ordering v, unspecified) noexcept;

 Returns: v

constexpr strong_ordering operator<=>(unspecified, strong_ordering v) noexcept;

 Returns: v < 0 ? strong_ordering::greater : v > 0 ? strong_ordering::less : v
```

6. Feature-testing recommendation

For the purposes of SG10, we recommend increasing the value of the macro attached to consistent comparisons (if any) to match date of acceptance of this proposal.

7. Implementability

At the following link, an example implementation of the comparison category types may be found - its goal is to reduce amount of branches:

- conversions between type categories are implemented by passing unmodified integer values,
- inversions for ordering types are implemented as negation of underlining integer value,
- with the single exception of `<=` and `>=` for `partial_ordering`, comparison between comparison category and `0` are implemented using single integer comparison,
- `<=` and `>=` for `partial_ordering` are implemented as disjunction of `<` and `==`.

This code can be tested online here — due lack of the language support, the declarations and uses of `operator<=>` are replaced with `operator_cmp` function.

8. Acknowledgements

Jens Maurer and Andrzej Krzemieński offered many useful suggestions and corrections to the proposal.

9. References

1. Herb Sutter, Jens Maurer, Walter E. Brown, "Consistent comparison" (P0515R3, http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0515r3.pdf)
2. Walter E. Brown, "Library Support for the Spaceship (Comparison) Operator" (P0768R1, http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0768r1.pdf)
3. Barry Revzin, "Implementing the spaceship operator for optional" (https://medium.com/@barryrevzin/implementing-the-spaceship-operator-for-optional-4de89fc6d5ec)
4. Richard Smith, "Working Draft, Standard for Programming Language C++" (N4713, http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4713.pdf)
5. Tomasz Kaminski, "Example implementation of comparision category types", (https://raw.githubusercontent.com/tomaszkam/proposals/master/implentation/space.cpp)